

RICHARD BOZEMAN

Senior Software Engineer at Saritasa

richard.bozeman.1992@outlook.com | linkedin.com/in/richard-bozeman-0a8b9a3a6 | Florida, United States | +1 (305) 7070624

SUMMARY

Senior Software Engineer who ships reliable software, improves systems, and occasionally argues with CI/CD until it admits defeat. I build products and platforms end-to-end: backend services, APIs, cloud infrastructure, observability, and the glue that keeps everything from falling apart at 2 AM. I'm big on clean architecture, pragmatic engineering, and making systems boring (the highest compliment in production). What you can expect from me: scalable services and well-designed APIs, cloud + DevOps mindset (automation over heroics), performance, security, and reliability baked in, strong ownership, clear communication, zero drama, mentoring, reviews, and helping teams level up. If your team likes shipping, learning, and laughing a little while doing serious work—let's connect.

SKILLS

Python 3.x (asyncio, typing, packaging, performance profiling)	API Design (REST, versioning, pagination, idempotency, backward compatibility)	Web Frameworks (FastAPI, Flask; middleware, dependency injection patterns)	AuthN/AuthZ (OAuth2 flows, JWT, RBAC/ABAC, secure session design)
Data Access (SQLAlchemy, migrations, transactional boundaries, query tuning)	Distributed Systems Basics (timeouts, retries, circuit breakers, bulkheads)	Background Processing (Celery-style patterns, scheduled jobs, poison message handling)	Caching (Redis patterns, cache invalidation strategies, request coalescing)
Testing (pytest, contract tests, integration harnesses, test data builders)	Security Practices (input validation, secrets management, least privilege)	Reliability (graceful degradation, backpressure, failure-path hardening)	Documentation (OpenAPI/Swagger, runbooks, architectural decision records)
React (component architecture, hooks, performance optimization, memoization)	TypeScript for React (strict typing, reusable generics, safer refactors)	State Management (context patterns, reducer-driven state, query caching patterns)	SPA Architecture (routing, auth guards, error boundaries, code splitting)
API Integration (typed clients, retry logic, optimistic updates, polling strategies)	UI Quality (accessibility basics, keyboard navigation, form validation patterns)	Testing (React Testing Library patterns, component contracts, mocking boundaries)	Build & Tooling (Vite/Webpack concepts, linting, formatting, CI checks)
Microsoft Azure (App Services concepts, storage, networking basics, identity integration)	Containers (Docker image hardening, multi-stage builds, runtime constraints)	CI/CD (pipeline design, gated releases, environment parity, rollback strategies)	Infrastructure Automation (repeatable environments, config-as-code mindset)
Observability (structured logging, metrics, traces; alerting and SLO-style thinking)	Operational Readiness (on-call runbooks, incident triage, post-incident follow-ups)	Performance & Cost Hygiene (right-sizing, latency budgets, noisy-neighbor mitigation)	Clean Architecture (boundary design, dependency direction, maintainable modules)
Code Reviews (signal over noise, actionable feedback, mentoring through reviews)	Incremental Refactoring (risk-managed changes, strangler patterns where appropriate)	Stakeholder Communication (scope clarity, tradeoffs, written updates, calm escalation)	Quality Culture (definition of done, test discipline, predictable delivery habits)
Collaboration (product/design/QA alignment, pragmatic planning, zero-drama teamwork)			

EXPERIENCE

Senior Software Engineer

Saritasa

May 2021 - November 2025 | California, United States

- Led end-to-end delivery for multiple product areas by turning ambiguous requests into crisp technical scopes, detailed acceptance criteria, and release plans that reduced churn and rework across teams.

- Designed and shipped Python-based APIs with consistent versioning, pagination, and idempotency rules, making integrations predictable for frontend and partner consumers even during frequent releases.
- Built React user flows that favored durable component boundaries and predictable state transitions, reducing UI regressions and making complex forms and dashboards easier to extend safely.
- Hardened failure paths by standardizing timeouts, retries, and defensive defaults, which reduced incident frequency and kept degraded-mode behavior understandable during partial outages.
- Improved performance by profiling hot paths, tightening queries, and applying caching where it actually helped, resulting in faster endpoints and fewer spikes during peak traffic windows.
- Strengthened release safety through CI checks, gated deployments, and clearer rollback procedures, lowering deployment anxiety and preventing “Friday night surprises” in production.
- Implemented structured logging and operational dashboards that made production debugging less guessy, enabling faster root-cause isolation and cleaner handoffs during on-call escalations.
- Raised code quality through pragmatic review standards and refactoring plans that targeted the most expensive pain points first, keeping delivery moving while steadily reducing technical debt.
- Collaborated closely with product and design to clarify edge cases early, preventing late-stage surprises and ensuring shipped behavior matched real-world workflows instead of idealized diagrams.
- Owned integration work across internal services and third-party systems by defining contracts and validation rules, reducing silent data drift and improving observability when inputs were messy.
- Guided junior and mid-level engineers with hands-on pairing and review feedback that focused on reasoning, boundaries, and tradeoffs, not just style, improving team autonomy over time.
- Introduced safer configuration practices by separating environment concerns and validating startup configuration, preventing misconfig-related outages and shortening diagnosis when issues happened.
- Delivered production-ready features with a bias for boring reliability, choosing proven patterns over novelty so systems remained stable even as business requirements shifted quickly.
- Handled incident response with calm, documented triage steps and post-incident follow-ups that produced actionable fixes, not blame, and measurably reduced repeat incidents.
- Communicated progress and risks clearly in writing, aligning stakeholders on scope tradeoffs and ensuring that expectations stayed realistic without slowing down delivery momentum.

Software Engineer

High Touch Technologies

December 2018 - April 2021 | Kansas, United States

- Delivered backend features and integrations with an emphasis on correctness, using clear validation rules and defensive error handling so production behavior stayed consistent under high change.
- Built and maintained APIs that supported UI and partner needs, improving response consistency and documentation habits so consumers could integrate without relying on tribal knowledge.
- Reduced manual release steps by tightening CI/CD workflows and adding automated checks, making deployments more repeatable and significantly lowering time spent on release-day firefighting.
- Improved production stability by diagnosing recurring failure patterns and applying targeted fixes, including better input handling and safer defaults for edge-case scenarios.
- Collaborated with QA and product to refine acceptance criteria and test coverage for high-risk areas, which reduced regressions and shortened the feedback loop on critical fixes.
- Enhanced data reliability by adding consistency checks and clearer logging around integration boundaries, making it easier to trace where mismatches or unexpected values originated.
- Contributed to frontend-facing work by aligning API contracts with UI needs, ensuring responses were shaped for real user workflows rather than forcing fragile client-side workarounds.
- Implemented practical monitoring and alerting signals that prioritized user-impacting failures, reducing noise and helping on-call focus on the alerts that actually mattered.
- Refactored brittle modules incrementally while still shipping features, keeping risk under control and steadily improving maintainability without pausing delivery for a “big rewrite.”
- Supported incident triage with reproducible steps and clear communication, enabling faster fixes and smoother coordination when multiple people were involved in diagnosis.
- Documented runbook-style notes for recurring operational tasks, turning one-off knowledge into repeatable processes that helped the team respond faster during pressure.

Software Developer

Webiz Digital

August 2016 - November 2018 | California, United States

- Built and maintained application features with a stability-first mindset, validating edge cases and ensuring new behavior integrated safely into existing user workflows without surprises.
 - Implemented backend functionality and API enhancements with consistent validation and error handling, reducing downstream issues caused by unexpected inputs and unclear contract behavior.
 - Worked closely with designers and stakeholders to translate requirements into concrete deliverables, keeping scope clear and minimizing late rework caused by misunderstandings.
 - Improved performance and reliability by tightening data access patterns and reducing unnecessary round trips, helping pages and endpoints feel snappier under real customer usage.
 - Supported production issues by reproducing bugs quickly and delivering targeted fixes, emphasizing root-cause thinking so the same class of problem did not keep returning.
 - Introduced more disciplined testing and review habits within day-to-day work, which improved confidence in changes and reduced the number of regressions shipped to users.
 - Contributed to deployment readiness by documenting environment assumptions and operational steps, making releases less dependent on one person remembering every detail.
-

Junior Software Developer

Minimalist Moon

September 2014 - August 2016 | Maine, United States

- Supported feature development and maintenance by implementing changes carefully and validating behavior against real workflows, helping keep day-to-day operations stable for end users.
- Diagnosed defects by isolating reproduction steps and instrumenting targeted logging, enabling faster fixes and building strong habits around clarity and accountability in technical work.
- Assisted with backend enhancements and small API improvements, focusing on consistent input validation and predictable error responses so client behavior remained dependable.
- Collaborated with senior engineers to learn clean boundaries and refactoring discipline, applying incremental improvements that made modules easier to understand and safer to change.
- Improved reliability by addressing common edge cases and tightening defensive checks, reducing the number of "it worked yesterday" failures triggered by unusual user inputs.
- Helped maintain build and release routines by following documented steps and improving documentation where gaps existed, making releases more repeatable for the whole team.
- Worked directly with feedback from support and stakeholders to prioritize fixes, ensuring that changes solved the real user pain instead of only addressing symptoms.
- Strengthened team velocity by taking ownership of small-but-important tasks end to end, freeing senior engineers for deeper work while still delivering measurable value.
- Developed a practical production mindset early by verifying changes in realistic environments, writing clearer notes, and communicating risks before they became incidents.

EDUCATION

Bachelor's degree in Computer Science

University of Pennsylvania

September 2010 - August 2014